

Synergizing LLMs and Formal Methods for Neural-Symbolic Reasoning

JIN SONG DONG, National University of Singapore, Singapore

YUFAN CAI, National University of Singapore, Singapore

ZHE HOU, Griffith University, Australia

JING SUN, University of Auckland, New Zealand

HONGSHU WANG, National University of Singapore, Singapore

YEDI ZHANG, National University of Singapore, Singapore

XINYUE ZUO, National University of Singapore, Singapore

Large language models (LLMs) have demonstrated remarkable capabilities in natural language understanding, generation, and reasoning, but their outputs remain probabilistic and error-prone. Formal methods (FM), by contrast, provide mathematically rigorous reasoning, but suffer from limited accessibility. This article explores a neural-symbolic paradigm that unifies these complementary strengths and presents case studies that illustrate how LLMs can lower the entry barrier to formal methods while the latter provide reliability, interpretability, and guarantees absent in purely neural AI models. We argue that synergizing LLMs with formal methods is a promising direction for building AI systems that are not only powerful but also trustworthy by construction.

CCS Concepts: • **Software and its engineering** → **Formal software verification**; **Automatic programming**; • **Theory of computation** → **Verification by model checking**; **Automated reasoning**; • **Computing methodologies** → **Natural language generation**; **Intelligent agents**.

Additional Key Words and Phrases: Large Language Model, Formal Method

ACM Reference Format:

Jin Song Dong, Yufan Cai, Zhe Hou, Jing Sun, Hongshu Wang, Yedi Zhang, and Xinyue Zuo. 2025. Synergizing LLMs and Formal Methods for Neural-Symbolic Reasoning. *J. ACM* 00, 0, Article 000 (2025), 10 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

In recent years, the rapid proliferation of large language models (LLMs) has marked a transformative shift in the landscape of artificial intelligence and many of its applications: software engineering [12], finance [28], healthcare [13] and legal [14], among others. Top LLMs, such as ChatGPT, Claude, Gemini and DeepSeek have demonstrated impressive capabilities in multi-modal understanding, generation, coding, task execution, and reasoning. Leveraging these capabilities, *agentic* LLMs can plan, take actions, and use feedback loops with memory to achieve goals autonomously.

Authors' Contact Information: Jin Song Dong, dcsdjs@nus.edu.sg, National University of Singapore, Singapore; Yufan Cai, National University of Singapore, Singapore, cai_yufan@u.nus.edu; Zhe Hou, Griffith University, Brisbane, Australia, z.hou@griffith.edu.au; Jing Sun, University of Auckland, Auckland, New Zealand, jing.sun@auckland.ac.nz; Hongshu Wang, National University of Singapore, Singapore, hongshu.wang@u.nus.edu; Yedi Zhang, National University of Singapore, Singapore, yd.zhang@nus.edu.sg; Xinyue Zuo, National University of Singapore, Singapore, zuoxy@nus.edu.sg.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-735X/2025/0-ART000

<https://doi.org/XXXXXXX.XXXXXXX>

Despite these advances, LLM reasoning suffers from a fundamental limitation: the tendency to produce factually incorrect or logically inconsistent yet plausible outputs, commonly termed hallucinations [11]. This poses serious risks in high-stakes domains where trustworthiness is critical. Importantly, this issue is not anomalous but mathematically inherent to the probabilistic objectives underlying LLM training. As an example, we asked the LLMs to solve the simple puzzle below: *In 318, only one digit is correct, and it is in the correct position. In 379, only one digit is correct, but it is in the wrong position. In 863, two digits are correct, but they are in the wrong positions. In 421, no digits are correct. What's the correct combination of the 3 digits and their positions?* Older models such as GPT 4.1, Gemini 2.5, Claude 4 Sonnet, DeepSeek V3 and Grok 3 gave wrong answers with believable thinking processes. Reasoning models such as the “Thinking” or “Pro” modes fared better but took minutes to respond. Small, open-source LLMs performed poorly and often fell into an infinite loop of reasoning. These models are constantly evolving — the latest versions of GPT, Gemini and Claude all answered correctly (698). However, hallucination persists theoretically and cannot be eradicated by training better LLMs alone [26].

Interestingly, we also asked Claude 4 to translate the puzzle into Z3’s SMT-LIB format, which Z3 solved correctly in 0.01 seconds *on a laptop* (MacBook Pro with M1 Pro). Z3 is a satisfiability modulo theories (SMT) solver [7], offering rigorous reasoning complementary to the probabilistic nature of LLMs. More broadly, formal methods (FM) provide mathematically precise tools for trustworthy computing: SMT extends classical *satisfiability* to theories such as arithmetic, bit-vectors, arrays, datatypes, and uninterpreted functions; *theorem proving* derives valid formulas from axioms via proof calculi; and *model checking* systematically explores system states to detect property violations.

Compared to the uncertain and generative nature of LLMs, formal methods deliver rigorous correctness guarantees, but their adoption is limited by steep learning curves and high expertise requirements. A promising direction is to unify these paradigms by leveraging LLMs for generation and interaction while using formal methods to enforce correctness and interpretability [6, 25].

2 A Neural-Symbolic Reasoning Paradigm

Notable neural-symbolic AI methods. A common early pattern in LLM-based formal reasoning is loosely coupled integration, where an LLM translates a problem into a formal language for a symbolic solver, and solver failures due to syntactic or semantic issues are fed back to the LLM for iterative refinement [2, 19, 21]. In contrast, tightly integrated approaches embed neural reasoning directly into the formal reasoning loop. For example, LeanDojo employs retrieval-augmented premise selection based on the current proof state and an encoder-decoder transformer to propose the next proof tactic, enabling fine-grained neural-symbolic collaboration [27].

Other tightly integrated paradigms include Logical Neural Networks (LNNs), where neurons correspond to components of weighted logical formulas in a real-valued logic, yielding end-to-end differentiability, omnidirectional inference, and training-time enforcement of logical consistency [20]. Scallop further advances this line through a neurosymbolic programming language and compiler that supports data- and computation-efficient training while delivering strong performance, interpretability, and generalization across diverse tasks [15]. Beyond such architectures, directly integrating formal methods into the internal components of LLMs remains possible but prohibitively resource-intensive. Our work instead adopts a practical and complementary strategy: constraining, checking, and guiding LLM outputs via external formal methods without requiring access to model internals. By formalizing the representations, constraints, and reasoning interfaces between LLMs and downstream systems, we ensure verifiability while enabling accessible deployment of reliability mechanisms in high-stakes applications.

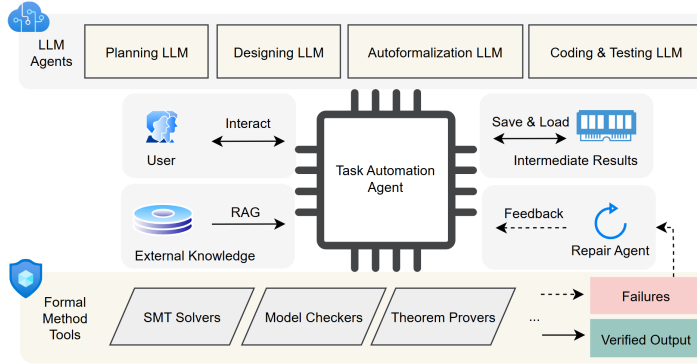


Fig. 1. An overview of our neural-symbolic reasoning framework.

Our paradigm. Across integration styles, common neural-symbolic interaction themes emerge (Fig. 1). To lower the entry barrier, natural language is used as input, with multiple LLM agents coordinated by an agentic *Task Automation LLM* that selects agents, interacts with users, and automates the workflow. Complex problems typically follow a “plan-execute” decomposition [12]. The resulting gap between informal input and formal specifications is bridged by an *Auto-formalization LLM*, while a *Designing LLM* converts underspecified prompts into high-level designs. Given the close link between formal methods and software engineering, many execution steps involve programming and are handled by a *Coding LLM*. Retrieval-augmented generation (RAG) supplies domain knowledge, including language syntax and semantics, and enables downstream applications. LLM agents interact with formal reasoning engines, including SMT solvers, theorem provers, and model checkers, whose reliable outputs provide high-quality feedback for repairing and refining auto-formalized code.

Guidelines on Architecture Design. An LLM+FM architecture should be selected from the *assurance target* outward, rather than from an LLM workflow inward. The aim is not merely better average outputs, but an architecture where success and failure are *observable* through formal feedback and where completion is gated by a *certificate*, i.e., machine-checkable evidence produced by the FM backend (e.g., a localized conflict explanation, a violating execution trace, or an undischarged proof). This also future-proofs the design: LLM capabilities may change quickly, but the acceptance boundary is defined by certificates that can be validated independently of the LLM.

Design Rule 1 (Assurance target): State the artifact that must be correct and the related properties that define correctness. Write the acceptance claim in the form “this output is acceptable because <property> holds for <artifact>.” If this claim cannot be stated precisely, later choices drift toward optimizing fluency or convenience rather than the intended notion of correctness.

Design Rule 2 (Backend FM choice): An FM backend should be chosen to match the property class and, more fundamentally, the domain of quantification. For constraint satisfiability and consistency questions that expect definitive outcomes (e.g., SAT/UNSAT), SMT-based backends are most appropriate, as they determine logical formula satisfiability and can return unsat cores to localize conflicting constraints, providing constraint-level correctness. For behavioral and temporal properties, model checking is better suited, as it reasons exhaustively over transition systems, verifies whether all executions satisfy a temporal specification, and otherwise produces counterexample traces, with first-class support for safety, liveness, and fairness, yielding behavioral/temporal correctness. For refinement and high-assurance correctness, theorem proving and refinement checking adopt a proof-centric approach, generating and discharging proof obligations to certify that a system meets its specification by construction, providing semantic, unbounded correctness; when full

certification is too costly, lighter-weight techniques such as type systems, abstract interpretation, or contract checking can offer faster but partial guarantees.

Design Rule 3 (LLM–FM interface): After fixing the FM backend, define the contract between generation and validation by deciding what the LLM may propose (candidates, decompositions, repairs) and what the FM tool must return on failure. Treat formal failures as typed signals rather than generic “try again” events, so prompts and repair logic are organized around channels such as inconsistency, counterexample, or unsatisfied obligation. Finer-grained diagnostics typically shorten repair cycles and reduce silent failure, at the cost of tighter tool integration and more orchestrated iterations. A further trade-off is where human effort is spent: it can be front-loaded via structured inputs that reduce ambiguity, or back-loaded via flexible inputs plus more validation cycles and occasional interactive confirmation.

Design Rule 4 (Integration pattern): Select a coupling pattern that matches the dominant failure mode *and* the task abstraction level. For decision/compliance settings (e.g., access-control policies, configuration constraints), *consistency-guarded deliberation* works well: deliberate freely, but gate acceptance with a consistency checker that surfaces localized conflicts. For system-level behaviors (e.g., protocol workflows, safety controllers), *counterexample-guided refinement* is effective: failed checks yield executions that drive targeted revisions, at the cost of repeated checking and occasional abstraction. For component synthesis (e.g., critical parsers and code), *proof-obligation guided synthesis* is effective: obligations pinpoint repair targets, but require precise specifications and manageable proof burden. Patterns are often combined across layers; when both system- and code-level assurance are needed, certify the higher-level model or contracts first, then constrain and certify lower-level synthesis.

Design Rule 5 (Acceptance guard): State explicitly the certificate that gates acceptance for the end-to-end system and what exactly it certifies. This operationalizes “good vs. bad” choices and makes trade-offs legible: stronger certificates raise iteration cost and may narrow applicability, while lighter-weight certificates broaden scope but leave more residual uncertainty. In practice, designs are most robust when they align the architecture with the task abstraction level, treat FM as the correctness filter rather than an optional add-on, and keep a clean separation between candidate generation and certificate-based validation.

In summary, LLMs and FM will form a complementary and beneficial generate–validate loop in future applications: LLMs explore the design space by proposing candidate artifacts, while FM backends enforce a certificate-based acceptance boundary grounded in explicit assurance targets. Formal failures verified by FM would become structured feedback that guides iterative refinement, aligning probabilistic generation with semantic guarantees. We will give some examples of our neural-symbolic reasoning paradigm in the following sections to demonstrate different roles of LLMs and formal reasoning in a variety of application domains.

3 FM4LLM: Legal AI with LLM-SMT Integration

As our first case study, we show how formal reasoning enhances the reliability and consistency of LLM outputs in legal reasoning. While substantive rationality concerns fairness and morality, formal rationality demands logical adherence to the law. Existing legal LLMs, such as LawLLM [22], primarily rely on natural language processing and retrieval-augmented generation to reduce hallucinations, but still lack logically grounded and reliable reasoning.

Our approach [4], as shown in Figure 2, addresses the above issue using SMT solving. Each statutory clause c is parsed into a typed rule $\text{Rule}(c) := \langle \text{Actor}, \text{Action}, \text{Cond}, \text{Penalty} \rangle$, and compiled into a first-order implication $\varphi_c := \forall x. \text{Actor}(x) \wedge \text{Action}(x) \wedge \text{Cond}(x) \rightarrow \text{Penalty}(x)$. All articles form the static legal knowledge base $\mathcal{K} := \{\varphi_c \mid c \in \text{Statutes}\}$. Given a case D , the Prosecutor and Attorney LLMs generate $(\mathcal{F}_P, \mathcal{S}_P) := \text{ProsecutorLLM}(D)$, $(\mathcal{F}_D, \mathcal{S}_D) := \text{AttorneyLLM}(D)$, where $\mathcal{F}$

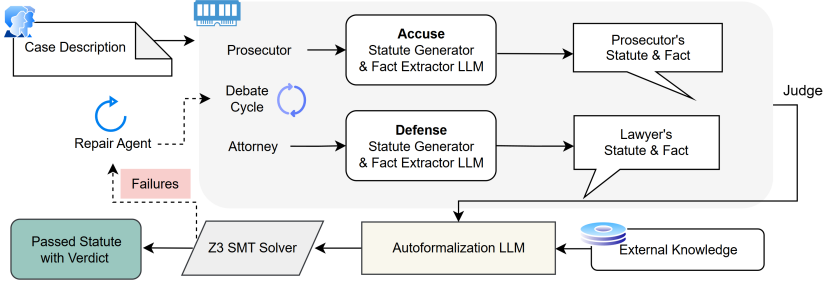


Fig. 2. The framework of our Legal LLM Agent.

are fact predicates and $\mathcal{S}$ are cited clauses. The Auto-formalizer LLM acts neutrally and takes the evidence provided by the above two LLMs and generates SMT-LIB format formal code that Z3 can check. If Z3 reports unsatisfiable, we will extract the *minimal unsatisfiable core*, representing the simplest reason that causes logical inconsistency. The Auto-formalizer LLM maps this core back to the original text and instructs the Prosecutor or Attorney LLM to drop the contradictory clauses, then asks Z3 to check for consistency again.

Example. Consider the following real case: (*Gong Qiang concerns a repeat narcotics offender*) Defendant Gong Qiang received a WeChat message from Luo, requesting the purchase of 1,000 yuan worth of drugs. After receiving the 1,000 yuan transfer from Luo via WeChat, Gong Qiang delivered 2 grams of methamphetamine and six magu tablets to Luo at approximately 5:00 PM ...

Taking the case information as input, the Prosecutor LLM (accusatory stance) extracts:

Actors: Defendant Gong Qiang (prior drug convictions), buyer Luo; public prosecution by Donghu District People's Procuratorate.

Acts: Sold 2 g methamphetamine + 6 "magu" tablets for 1 000 RMB.

Evidence: WeChat payment record, seized drugs, positive urine tests, confession.

Aggravators: Repeat drug offender in five-year window; multiple prior administrative penalties.

Statutory Law: [64, 65]; [347, 356].

On the other hand, the Attorney LLM (exculpatory stance) extracts:

Actors: Defendant Gong Qiang, buyer Luo; defense counsel appointed.

Acts: Acknowledge sale but stress "very small quantity" and no wider circulation.

Evidence: WeChat record, drug test, confession.

Mitigators: Full confession, signed plea agreement, voluntary compensation and buyer's understanding; prior sentence already served, limited social harm.

Statutory Law: [64, 67]; [347, 356].

The Auto-formalizer LLM analyzes the above results and further extracts the following facts and then converts them into SMT-LIB format and then sends them to the Z3 solver together with the extracted rules and clauses:

- Article 65: matched field { prior_sentence: True, reoffense_in_5_years: True }
- Article 67: matched field { truthful_confession: True }
- Article 347: matched fields {subject: person, action: traffic_narcotics, object: narcotics, intent: intentional}

Z3 evaluates the logical constraints and produces consistent results, which a judge LLM converts into a human-readable verdict of two years' imprisonment and a 10,000 yuan fine, close to the ground truth of one year and two months with an 11,000 yuan penalty. This legally informed integration employs multiple role-specific LLM agents coordinated with an SMT solver to ensure

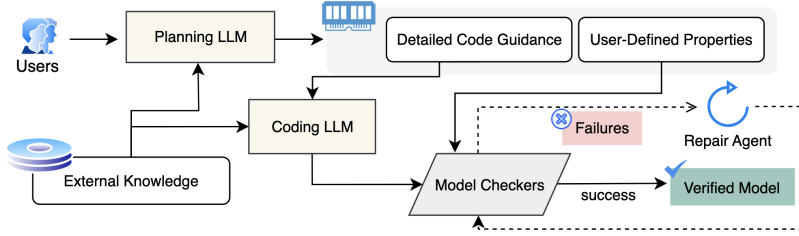


Fig. 3. An overview of our LLM model checking agent framework.

logical consistency, leading to more realistic sentencing and the highest accuracy among state-of-the-art methods [4].

4 LLM4FM: LLM Model Checking Agent

LLMs can reduce the costly manual effort in formal methods. This case study presents a multi-LLM-agent framework that automates the full model-checking workflow. While model checking itself is automated once a formal model and properties are provided, its industrial adoption is hindered by the manual creation of models and specifications, which requires expertise in checker-specific modeling languages and temporal logics unfamiliar to most engineers.

Auto-formalization is central to LLM-driven formal methods. Prior work has used LLMs to translate natural language into Linear Temporal Logic (LTL) formulas [9] and to synthesize or repair formal proofs [8], but these do not target system-level modeling. Our case study focuses on the Process Analysis Toolkit (PAT) [23], which employs CSP# [10] as the modeling language. The PAT-Agent framework [29] (Fig. 3) takes a natural-language system description comprising system behavior and expected properties, translating the former into a formal model and the latter into requirements—properties with specified expected outcomes (i.e., expected to hold or not hold).

We introduce the following notation: Let $\mathcal{L}_{\text{NL}}$ be the space of natural-language descriptions, Π the planning representations, $\mathcal{P}$ the generation plans, $\mathcal{M}$ the PAT models, $\mathcal{Q}$ the user requirements, $\mathcal{V}$ PAT’s verification procedure, $\mathcal{C}$ the counterexample traces, and $\mathcal{R}$ the repair directives. With these notations, our pipeline is summarized by the four transformations:

$$\mathcal{T}_{\text{plan}} : \mathcal{L}_{\text{NL}} \rightarrow \Pi \rightarrow \mathcal{P}; \quad \mathcal{T}_{\text{gen}} : \mathcal{P} \rightarrow \mathcal{M}; \quad \mathcal{V} : \mathcal{M} \times \mathcal{Q} \rightarrow \{\text{MATCH}, \text{MISMATCH}\}^m \times \mathcal{C}; \quad \mathcal{T}_{\text{repair}} : \mathcal{M} \times \mathcal{C} \times \mathcal{R} \rightarrow \mathcal{M}.$$

Our objective is to obtain a model $M^* \in \mathcal{M}$ such that $\forall (\varphi, o) \in \mathcal{Q} : \mathcal{V}(M^*, \varphi) = o$. In the first stage, a Planning LLM parses the system description to extract semantic elements such as constants, variables, actions, and guards, and produces a model-generation plan that defines the intended structure and includes placeholder annotations with explanatory comments to guide subsequent code synthesis. It also processes the user-specified requirements for the system (e.g., reachability and liveness requirements).

User studies show that PAT-Agent translates requirements with 99.6% accuracy across multiple projects. In rare cases, manual checks are still required, supported by optional checkpoints in PAT-Agent. In the second stage, a Coding LLM translates the model-generation plan into CSP# code, guided by formal syntax and semantics and a corpus of existing PAT models. In our experiments, Claude 3.7 Sonnet generates CSP# code with near-perfect syntax on the first attempt, with any minor issues resolved automatically through brief LLM interaction. Full semantic correctness cannot be guaranteed in an automated method, as it depends on user intent, but this issue can be mitigated by checking the generated model against the automatically formalized properties and expected outcomes using PAT. Assuming that the properties are formalized correctly and they sufficiently capture the user’s intent, model checking itself will largely resolve modeling inaccuracies. When mismatches occur, a model repair loop is triggered: PAT’s output, including counterexamples,

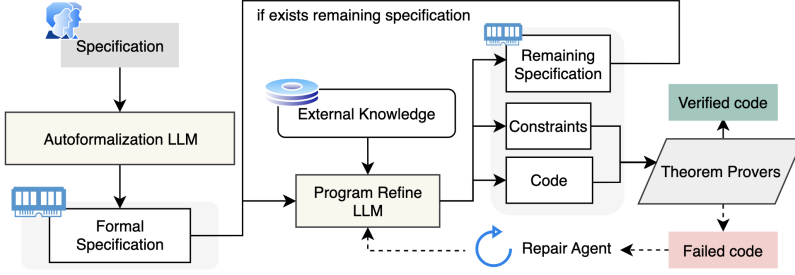


Fig. 4. An integration of LLM and program refinement for reliable code generation.

guides a Repair Agent to identify and fix only the faulty code region rather than regenerating the entire model. In practice, three to four repair iterations typically suffice to satisfy all properties.

Example. An electrical engineer describes the motor engine as follows¹:

This process models a car’s motor. The engine can be turned on and off, the car can start and stop driving, the fuel is gradually consumed after driving, and it can be refueled.

The Planning LLM extracts and defines key constants, variables, and actions (e.g., carMotion, fuel, engineStatus, startDriving), assigning context-appropriate names and inferring implicit system logic using common sense. For example, it deduces that startDriving requires the engine to be on, the car to be stationary, sufficient fuel, and driver presence. The Coding LLM then instantiates this plan by filling in concrete model details, translating such inferred actions into:

```
[engineStatus == engineOn && carMotion == stop && (fuel > 0) && driver == inCar]
startDriving {carMotion = drive;} -> motor()
```

The code follows CSP# syntax [conditions] event_name data_operations -> next_process(), where guards specify enabling conditions and data operations define event effects, and it passes PAT’s syntax check on the first attempt. The Coding LLM also formalizes system requirements, for example, translating “the car should not be in driving mode without the key” into a reachability assertion, where the state driveWithoutKey is expected to be *unreachable*.

```
#define driveWithoutKey (carMotion==drive && key!=inCar);
#assert car reaches driveWithoutKey; // Expects Invalid
```

However, verification results may show that the above state *is reachable* given the initial model generated by the Coding LLM. This issue, together with a counterexample trace composed of system states that lead to the error, is sent to the Repair Agent for a repair iteration, after which the above process is revised as follows to avoid the error:

```
[engineStatus == engineOn && carMotion == stop && (fuel > 0) && driver == inCar
&& key == inCar] startDriving {carMotion = drive;} -> motor()
```

Interestingly, this issue reflects a real-life situation in which the author of this article kept the engine on and dropped off a passenger at a shopping mall who took the key, and that author continued to drive the car to the office, only to realize that he could not lock the car! With PAT-Agent, automakers can now verify their designs without the excuse of “But we do not have formal methods engineers.”

5 They Are One: Trustworthy AI Coder

Some powerful integrations of neural-symbolic reasoning require the two parts to work hand-in-hand in each small step. This case study demonstrates an AI coder that combines LLM, program

¹Note that a complete system description and model can be lengthy, so we only extract a small part in this example.

refinement and theorem proving into a hybrid system. LLMs have shown potent coding capabilities, and the latest ones have raised the bar to the project level — the user only needs to give a one-line prompt, and GPT 5 can build a fully functional website. However, the fundamental reliability problem remains: Is the code functionally *correct* with respect to user specifications?

Example. We asked GPT 5 to write the following function:

Generate a program that finds the square root of N within the error bound e .

GPT 5 produced the Python code on the right. However, the reader can check that the two lines after the for loop are incorrect as there exist N and e such that $\text{abs}(x_{\text{next}} - x) \leq e$ but $\text{abs}(x_{\text{next}}^2 - N) > e$, which violates the specification. Such issues prompted us to build a reliable AI coder [3], as illustrated in Figure 4. To ensure *correctness*, we adopt the refinement calculus [18] to derive code *from* first-order logic specifications via correctness-preserving transformations. A program is viewed as a predicate transformer, and refinement as logical implications between transformers. Starting from a pure specification **Spec**, the calculus incrementally replaces conjuncts and quantifiers with executable constructs such as assignments, guards, loops, and nondeterministic choices using laws like *assignment introduction*, *guard strengthening*, and *loop introduction*, until only code remains. Traditionally, each refinement step requires human intervention: (i) selecting a refinement law whose side conditions hold, (ii) instantiating schema variables to produce a program fragment, and (iii) discharging the proof obligations (POs) ensuring **Spec** $\sqsubseteq$ **Code**. This manual workflow scales poorly for real-world systems where large specifications may require hundreds of refinement steps. To address *scalability*, we integrate LLMs into the refinement loop. Given the current specification state S_i , the LLM (1) predicts a suitable refinement law, (2) instantiates it to generate a candidate code fragment C_i , and (3) emits the associated proof obligation (PO) induced by the refinement law. For example, the VC for an assignment, $\{P\} x := E \{Q\}$, requires proving the logical implication $P \Rightarrow Q[x \mapsto E]$. An automated theorem prover (ATP) attempts to discharge all POs. If all succeed, the refinement commits to S_{i+1} . If any PO fails, the ATP produces the failing proof state (e.g., an unsatisfied VC or counterexample model), which is fed back to the LLM. The LLM may revise the code, select a different refinement rule, or backtrack to the last verified state. This yields a search-and-repair refinement process jointly guided by symbolic reasoning and neural generation.

Back to the square root example, our method generated the code below. Our code is *guaranteed* correct with respect to the proof obligations required by the refinement calculus in each step. Only one human input remains: This method requires the specification in first-order logic at the starting point, but not every programmer is knowledgeable in logic.

LLM-based auto-formalization can mitigate this, as done in the previous case study, but the core issue remains: What if the translation is wrong? We argue that no perfect and fully automatic solution exists, as verification ultimately depends on correctness criteria defined by the user. In the experiment, our method improves the pass rate on popular datasets, such as HumanEval [5] and EvalPlus [16], over the base LLM. That is, if the LLM becomes more intelligent, our method's result will be improved on top of that. More importantly, unlike LLMs, whose pass rate decreases with more test cases, indicating that potential bugs still exist, our method's pass rate is consistent regardless of test cases because programs are

```
def sqrt_w_err(N, e, max_iter=10000):
    if e <= 0 or N < 0:
        raise ValueError
    if N == 0: return 0.0
    x = N if N >= 1 else 1.0
    for _ in range(max_iter):
        x_next = 0.5 * (x + N / x)
        if (abs(x_next - x) <= e
            or abs(x_next^2 - N) <= e):
            return x_next
    x = x_next
    return x
```

```
N, e = input()
x, y = 0, N+1
while y > x+e:
    if (x+y)/2*(x+y)/2 > N:
        y = (x+y)/2
    else: x = (x+y)/2
```

proven correct. Those that do not pass are mainly because the theorem prover timed out, which, in turn, could be enhanced by LLMs.

6 Mitigating Auto-formalization Risk

A common issue across LLM+FM paradigms is the translation from informal user intention to formal specification. We generally assume that the user sufficiently specifies their requirements in natural language and manually confirms the formal specification generated by the auto-formalization module. Beyond syntax, the main risk is semantic under-specification: natural language omits assumptions, scopes, and constraints, so an auto-formalizer can produce a logically consistent but intent-misaligned specification that still passes solver checks and yields certificates for the wrong target. Robustness can be improved with typed intermediate representations and constrained decoding, where the LLM emits a type-checked schema compiled deterministically into the target formalism, and with semantic validation via round-trip paraphrases and automatically generated “should/should-not” scenarios checked against both the original intent and the formal solver.

A practical mitigation is *N-version auto-formalization*: generate multiple independent formalizations using different LLMs or prompt/decoding variants of the same LLM, and filter them with lightweight formal checks (well-formedness/type constraints, satisfiability and non-triviality, and consistency against fixed background constraints). Material disagreement in structure or consequences is treated as a high-risk signal and surfaced explicitly rather than being silently resolved. When appropriate, a *router+specialists* strategy (a system-level mixture-of-experts) can further reduce mismatch by dispatching inputs to domain- and logic-specific formalizers or templates, while keeping the same certificate-producing backend as the acceptance gate.

Finally, while certificate-producing backends provide strong guarantees of logical validity, they do not eliminate epistemic uncertainty regarding specification fidelity. Formal methods act as verifiers of consistency, not arbiters of semantic truth. Bridging the informal–formal gap therefore remains an open research challenge, requiring improved interactive specification protocols, structure-aware validation, and human-in-the-loop confirmation mechanisms.

7 Conclusion and Future Work

This article presented a neural–symbolic paradigm in which LLMs propose candidate artifacts and formal methods gate acceptance with machine-checkable certificates. The three case studies illustrate this paradigm, but the key contribution is a reusable design view: select from the assurance target outward, choose an FM backend for the property class, and use formal feedback to drive revision so that errors become observable rather than implicit.

Beyond the case studies, the same paradigm applies to other integrations of LLM and FM [24] and applications in compliance analysis, protocol verification, cyber-physical safety, and synthesis of critical components. Recent progress around Lean further illustrates this opportunity. Mathlib [17] shows how an extensive machine-checked library can support large-scale formalization and provide a strong foundation for later AI-assisted proof tools, while CSLib [1] seek to extend this effort to computer science concepts and verified code for AI-aided FM. Together, these projects suggest that shared formal libraries and reusable proof artifacts can help LLMs and formal methods achieve results beyond what either could readily do alone.

Future work will broaden these directions and further close the auto-formalization gap via N-version translation (across models or prompt variants), router+specialists, typed intermediate representations, and scenario-based semantic validation.

References

- [1] Clark Barrett et al. 2026. CSLib: The Lean Computer Science Library. arXiv:2602.04846

- [2] Sahil Bhatia, Jie Qiu, Niranjan Hasabnis, Sanjit A Seshia, and Alvin Cheung. 2024. Verified code transpilation with llms. *Advances in Neural Information Processing Systems* 37 (2024), 41394–41424.
- [3] Yufan Cai, Zhe Hou, David Sanan, Xiaokun Luan, Yun Lin, Jun Sun, and Jin Song Dong. 2025. Automated Program Refinement: Guide and Verify Code Large Language Model with Refinement Calculus. *ACM Principles of Programming Languages* (2025).
- [4] Linze Chen, Yufan Cai, Zhe Hou, and Jin Song Dong. 2025. Towards Trustworthy Legal AI through LLM Agents and Formal Reasoning. arXiv:2511.21033 [cs.AI] <https://arxiv.org/abs/2511.21033>
- [5] Mark Chen et al. 2021. Evaluating Large Language Models Trained on Code. (2021). arXiv:2107.03374 [cs.LG]
- [6] David Dalrymple et al. 2024. Towards Guaranteed Safe AI: A Framework for Ensuring Robust and Reliable AI Systems. arXiv:2405.06624 [cs.AI] <https://arxiv.org/abs/2405.06624>
- [7] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 337–340.
- [8] Emily First, Markus Rabe, Talia Ringer, and Yuriy Brun. 2023. Baldur: Whole-proof generation and repair with large language models. In *The 31st ACM Conference on the Foundations of Software Engineering*. 1229–1241.
- [9] Francesco Fuggitti and Tathagata Chakraborti. 2023. NL2LTL—a python package for converting natural language (NL) instructions to linear temporal logic (LTL) formulas. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [10] Charles Antony Richard Hoare. 1978. Communicating sequential processes. *Commun. ACM* 21, 8 (1978), 666–677.
- [11] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *Comput. Surveys* 55, 12 (2023), 1–38.
- [12] Xue Jiang, Yihong Dong, Lecheng Wang, Zheng Fang, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. 2024. Self-planning code generation with large language models. *ACM Transactions on Software Engineering and Methodology* (2024).
- [13] Yubin Kim, Xuhai Xu, Daniel McDuff, Cynthia Breazeal, and Hae Won Park. 2024. Health-LLM: Large Language Models for Health Prediction via Wearable Sensor Data. In *Conference on Health, Inference, and Learning*. 522–539.
- [14] Haitao Li, Qingyao Ai, Qian Dong, and Yiqun Liu. 2024. LexiLaw: A Scalable Legal Language Model for Comprehensive Legal Understanding. <https://github.com/CSHaitao/LexiLaw>.
- [15] Ziyang Li, Jiani Huang, and Mayur Naik. 2023. Scallop: A Language for Neurosymbolic Programming. *ACM Programming Language Design and Implementation* (2023). doi:10.1145/3591280
- [16] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by ChatGPT really correct. *Rigorous evaluation of large language models for code generation*. CoRR, abs/2305.01210 (2023).
- [17] The mathlib Community. 2020. The lean mathematical library (*ACM CPP 2020*). 367–381. doi:10.1145/3372885.3373824
- [18] Carroll Morgan. 1990. *Programming from Specifications*. Prentice-Hall, Inc., USA.
- [19] Liangming Pan, Alon Albalak, Xinyi Wang, and William Wang. 2023. Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 3806–3824.
- [20] Ryan Riegel et al. 2020. Logical Neural Networks. arXiv:2006.13155 [cs.AI] <https://arxiv.org/abs/2006.13155>
- [21] Sanjit A Seshia. 2015. Combining induction, deduction, and structure for verification and synthesis. *Proc. IEEE* 103, 11 (2015), 2036–2051.
- [22] Dong Shu, Haoran Zhao, Xukun Liu, David Demeter, Mengnan Du, and Yongfeng Zhang. 2024. LawLLM: Law Large Language Model for the US Legal System. In *ACM International Conference on Information and Knowledge Management (CIKM '24)*. ACM, 4882–4889. doi:10.1145/3627673.3680020
- [23] Jun Sun, Yang Liu, Jin Song Dong, and Jun Pang. 2009. PAT: Towards Flexible Verification under Fairness. In *Computer Aided Verification*. Springer.
- [24] Hongshu Wang, Xinyue Zuo, Yuhan Sun, Qin Li, Yamine Ait Ameur, and Jin Song Dong. 2026. Event-B Agent: Towards LLM Agent for Formal Model Synthesis and Repair. In *ACM International Conference on the Foundations of Software Engineering*, 2026.
- [25] Jeannette M. Wing. 2021. Trustworthy AI. *Commun. ACM* 64, 10 (Sept. 2021), 64–71. doi:10.1145/3448248
- [26] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. 2024. Hallucination is Inevitable: An Innate Limitation of Large Language Models. arXiv:2401.11817 [cs.CL] <https://arxiv.org/abs/2401.11817>
- [27] Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J Prenger, and Animashree Anandkumar. 2023. Leandojo: Theorem proving with retrieval-augmented language models. *Advances in Neural Information Processing Systems* 36 (2023), 21573–21612.
- [28] Yi Yang, Yixuan Tang, and Kar Yan Tam. 2023. Investlm: A large language model for investment using financial domain instruction tuning. *arXiv preprint arXiv:2309.13064* (2023).
- [29] Xinyue Zuo, Yifan Zhang, Hongshu Wang, Yufan Cai, Zhe Hou, Jing Sun, and Jin Song Dong. 2025. PAT-Agent: Autoformalization for Model Checking. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2122–2133.